Edge Detection in Digital Images

From Gradient Operators to Canny and Modern Deep Edge Detection

Contents

1 Introduction and Motivation 4

2 What is an Edge? 4

3 Edge Orientation 5

4 From Continuous Derivatives to Discrete Images 5

5 Why Do We Need Larger Kernels? 5

6 Sobel Edge Detector 6

7 Deriving the Sobel Kernel 6

8 Why the Weights $1, 2, 1$? 7

9 Prewitt Operator 8

10 Prewitt vs. Sobel 8

11 Limitations of First-Order Operators 9

12 Second Derivative and the Laplacian 9

13 Discrete Laplacian 9

14 Why the Laplacian Alone is Problematic 10

15 Gaussian Smoothing 10

16 Laplacian of Gaussian 10

17 Deriving the LoG Equation 11

18 LoG Edge Detection 11

19 Interpretation of σ in LoG 12

20 From LoG to Canny 12

21 Canny Edge Detection: Overview 13

22 Step 1: Gaussian Smoothing 13

23 Step 2: Gradient Computation **13**

24 Step 3: Non-Maximum Suppression **14**

25 Gradient Direction Quantization **14**

26 Step 4: Double Thresholding **14**

27 Step 5: Edge Tracking by Hysteresis **15**

28 Canny: Complete Mathematical Pipeline **15**

29 How Canny Can Be Understood Mathematically **15**
29.1 Good Detection 16
29.2 Good Localization 16
29.3 Single Response 16

30 Relationship Between LoG and Canny **16**

31 Limitations of Classical Edge Detection **16**

32 Evolution of Edge Detection **17**

33 Structured Edge Detection **17**

34 Holistically-Nested Edge Detection (HED) **18**

35 Richer Convolutional Features **18**

36 Multi-Scale and Encoder–Decoder Edge Detection **18**

37 DexiNed **19**

38 Attention and Transformer-Based Edge Detection **19**

39 Classical vs. Learning-Based Edge Detection **20**

40 A Useful Conceptual View **20**
40.1 Problem 1: Detect Intensity Changes 20
40.2 Problem 2: Suppress Noise 20
40.3 Problem 3: Produce Thin and Connected Edges 20
40.4 Problem 4: Understand Which Changes are Meaningful 20

41 Summary **21**

42 Suggested Teaching Sequence **22**

43 Appendix **22**
43.1 Why Compare Along the Gradient Direction? 23
43.2 Idealized Mathematical Formulation 23
43.3 Discrete Implementation 23
43.4 Interpolation View of NMS 24

44 Double Thresholding **24**

45 Edge Tracking by Hysteresis **25**
45.1 Graph Interpretation of Hysteresis 25
45.2 Recursive Formulation 26

46 Why Hysteresis is Better Than a Single Threshold **26**

47 NMS and Hysteresis: Their Different Roles **26**

48 Complete Canny Pipeline with Mathematical Operations **27**

1 Introduction and Motivation

An image can be viewed as a two-dimensional function

$$I(x, y), \tag{1}$$

where $I(x, y)$ represents the intensity at spatial location (x, y).

Objects and regions in an image are often separated by boundaries where the intensity changes significantly.

For example, consider a simple one-dimensional intensity profile:

$$I(x) = \begin{cases} I_1, & x < x_0, \\ I_2, & x \geq x_0. \end{cases} \tag{2}$$

There is a sudden change in intensity at $x = x_0$.

This sudden change is an example of an **edge**.

The basic idea behind edge detection is therefore:

Find locations where the image intensity changes rapidly.

2 What is an Edge?

An edge is a local region of an image where the intensity changes significantly.

Consider a one-dimensional intensity profile:

$$I(x). \tag{3}$$

If the intensity changes slowly,

$$\frac{dI}{dx} \approx 0. \tag{4}$$

If the intensity changes rapidly,

$$\left|\frac{dI}{dx}\right| \gg 0. \tag{5}$$

Therefore, the first derivative provides a natural way of detecting edges.

In two dimensions, the equivalent quantity is the gradient:

$$\boxed{\nabla I = \begin{bmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{bmatrix}} \tag{6}$$

The magnitude of the gradient is

$$\boxed{\|\nabla I\| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}} \tag{7}$$

A large gradient magnitude indicates a likely edge.

3 Edge Orientation

The gradient does more than tell us whether an edge exists.

It also tells us the direction of maximum intensity change.

The gradient direction is

$$\boxed{\theta(x,y) = \tan^{-1}\left(\frac{I_y}{I_x}\right)} \tag{8}$$

where

$$I_x = \frac{\partial I}{\partial x}, \qquad I_y = \frac{\partial I}{\partial y}. \tag{9}$$

An important geometric observation is:

The gradient is perpendicular to the edge.

For example, if an edge is approximately vertical, the intensity changes primarily in the horizontal direction.

Therefore,

$$|I_x| \gg |I_y|.$$

4 From Continuous Derivatives to Discrete Images

The image is not a continuous function in a digital computer.

Instead, we have pixels:

$$I[i,j]. \tag{10}$$

Therefore, derivatives must be approximated using finite differences.

A simple forward difference is

$$\frac{\partial I}{\partial x} \approx I(x+1,y) - I(x,y). \tag{11}$$

This corresponds to the kernel

$$\boxed{K_x = \begin{bmatrix} -1 & 1 \end{bmatrix}} \tag{12}$$

Similarly,

$$\boxed{K_y = \begin{bmatrix} -1 \\ 1 \end{bmatrix}} \tag{13}$$

These are simple derivative operators, but they are highly sensitive to noise.

5 Why Do We Need Larger Kernels?

Consider an image containing noise:

$$I(x,y) = I_{\text{true}}(x,y) + n(x,y). \tag{14}$$

Differentiation amplifies high-frequency components.

Therefore, even small amounts of noise can result in large derivative responses.

We would like a filter that performs two operations:

1. Smooth the image to reduce noise.
2. Estimate the local derivative.

This motivates the use of larger derivative kernels such as Sobel.

6 Sobel Edge Detector

The Sobel operator uses two 3×3 kernels:

$$\boxed{G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}} \tag{15}$$

and

$$\boxed{G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}} \tag{16}$$

The filtered images are

$$I_x = G_x * I \tag{17}$$

and

$$I_y = G_y * I. \tag{18}$$

The gradient magnitude can then be estimated as

$$\boxed{G = \sqrt{I_x^2 + I_y^2}} \tag{19}$$

or, for computational efficiency,

$$\boxed{G \approx |I_x| + |I_y|}. \tag{20}$$

7 Deriving the Sobel Kernel

The Sobel kernel is not an arbitrary collection of numbers.
It can be understood as the combination of:

1. a derivative operation in one direction, and
2. a smoothing operation in the perpendicular direction.

Consider the one-dimensional derivative kernel

$$D = \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}. \tag{21}$$

This estimates the horizontal derivative.
However, we also want to smooth in the vertical direction.
A simple smoothing kernel is

$$S = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}. \tag{22}$$

The 2 in the middle gives greater weight to the central row.
The two-dimensional Sobel kernel is obtained by the outer product

$$G_x = SD. \tag{23}$$

Therefore,

$$G_x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}. \tag{24}$$

Multiplication gives

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}. \tag{25}$$

Thus,

$$\boxed{G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}} \tag{26}$$

Similarly,

$$G_y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}, \tag{27}$$

giving

$$\boxed{G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}}. \tag{28}$$

This is an important interpretation of Sobel:

$$\boxed{\text{Sobel} \approx \text{Derivative} \times \text{Smoothing}} \tag{29}$$

8 Why the Weights $1, 2, 1$?

The smoothing vector

$$\begin{bmatrix} 1 & 2 & 1 \end{bmatrix} \tag{30}$$

is closely related to a small discrete approximation to a Gaussian smoothing kernel.
Normalized,

$$\frac{1}{4} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} \tag{31}$$

acts as a weighted local average.

Therefore, compared with a simple derivative kernel, Sobel introduces some smoothing before estimating the derivative.

This improves robustness to noise.

9 Prewitt Operator

The Prewitt operator is another first-order derivative-based edge detector.

Its horizontal and vertical kernels are

$$\boxed{P_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}} \tag{32}$$

and

$$\boxed{P_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}}. \tag{33}$$

The corresponding responses are

$$I_x = P_x * I \tag{34}$$

and

$$I_y = P_y * I. \tag{35}$$

The gradient magnitude is

$$G = \sqrt{I_x^2 + I_y^2}. \tag{36}$$

10 Prewitt vs. Sobel

Both Prewitt and Sobel are based on first derivatives.

The main difference is the smoothing weights.

Prewitt uses

$$\begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \tag{37}$$

whereas Sobel uses

$$\begin{bmatrix} 1 & 2 & 1 \end{bmatrix}. \tag{38}$$

Thus, Sobel gives greater importance to the central row or column.

	Prewitt	**Sobel**
Derivative	Yes	Yes
Smoothing	Uniform	Weighted
Kernel size	3×3	3×3
Noise robustness	Moderate	Generally better

11 Limitations of First-Order Operators

Sobel and Prewitt have several limitations.

1. They are sensitive to noise.
2. They produce relatively thick edges.
3. Threshold selection is required to convert gradient magnitude into a binary edge map.
4. They do not explicitly perform edge localization.
5. Multiple nearby responses may be generated around a true edge.

This motivates more sophisticated approaches.
One important direction is to use the **second derivative**.

12 Second Derivative and the Laplacian

For a two-dimensional image $I(x, y)$, the Laplacian is defined as

$$\boxed{\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}} \tag{39}$$

The second derivative responds strongly to rapid changes in the first derivative.
Consider a one-dimensional ideal step edge.
The intensity changes rapidly at the edge.
The first derivative produces a peak at the edge.
The second derivative changes sign around the edge.
Therefore, an edge can be detected using a **zero crossing** of the second derivative.

13 Discrete Laplacian

A simple approximation to the second derivative is

$$\frac{\partial^2 I}{\partial x^2} \approx I(x+1, y) - 2I(x, y) + I(x-1, y). \tag{40}$$

Similarly,

$$\frac{\partial^2 I}{\partial y^2} \approx I(x, y+1) - 2I(x, y) + I(x, y-1). \tag{41}$$

Adding the two gives

$$\nabla^2 I \approx I(x+1, y) + I(x-1, y) + I(x, y+1) + I(x, y-1) - 4I(x, y). \tag{42}$$

This corresponds to the kernel

$$\boxed{L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}}. \tag{43}$$

Another commonly used approximation is

$$\boxed{L = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}}. \tag{44}$$

14 Why the Laplacian Alone is Problematic

The Laplacian is a second derivative.

Differentiation emphasizes high-frequency components.

Noise is also dominated by high-frequency components.

Therefore,

$$\boxed{\text{Second derivative} \quad \Rightarrow \quad \text{strong sensitivity to noise}} \tag{45}$$

A natural solution is:

Smooth first, then apply the Laplacian.

This leads to the **Laplacian of Gaussian (LoG)**.

15 Gaussian Smoothing

The two-dimensional Gaussian function is

$$\boxed{G(x, y; \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)} \tag{46}$$

where σ controls the amount of smoothing.

The smoothed image is

$$I_s = G * I. \tag{47}$$

The larger σ is, the stronger the smoothing.

16 Laplacian of Gaussian

The LoG operator applies the Laplacian to the Gaussian:

$$\boxed{LoG(x, y) = \nabla^2 G(x, y)} \tag{48}$$

Since convolution and differentiation can be interchanged,

$$\nabla^2 (G * I) = (\nabla^2 G) * I. \tag{49}$$

Therefore,

$$\boxed{LoG * I = \nabla^2 (G * I)} \tag{50}$$

This means that we can either:

1. smooth the image and then apply the Laplacian, or
2. convolve the image directly with the LoG kernel.

17 Deriving the LoG Equation

Start with

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}. \tag{51}$$

Let

$$r^2 = x^2 + y^2. \tag{52}$$

Then

$$G = \frac{1}{2\pi\sigma^2} e^{-\frac{r^2}{2\sigma^2}}. \tag{53}$$

We need

$$\nabla^2 G = \frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2}. \tag{54}$$

First,

$$\frac{\partial G}{\partial x} = -\frac{x}{\sigma^2} G. \tag{55}$$

Differentiating again,

$$\frac{\partial^2 G}{\partial x^2} = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right) G. \tag{56}$$

Similarly,

$$\frac{\partial^2 G}{\partial y^2} = \left(\frac{y^2}{\sigma^4} - \frac{1}{\sigma^2}\right) G. \tag{57}$$

Adding the two,

$$\nabla^2 G = \left(\frac{x^2+y^2}{\sigma^4} - \frac{2}{\sigma^2}\right) G. \tag{58}$$

Substituting the Gaussian expression gives

$$\boxed{LoG(x,y) = -\frac{1}{\pi\sigma^4}\left(1 - \frac{x^2+y^2}{2\sigma^2}\right) e^{-\frac{x^2+y^2}{2\sigma^2}}} \tag{59}$$

up to an overall sign depending on the convention used.

18 LoG Edge Detection

The LoG detector can therefore be understood as:

$$\boxed{I \xrightarrow{\text{Gaussian smoothing}} G * I \xrightarrow{\text{Laplacian}} \nabla^2 (G * I)} \tag{60}$$

or equivalently,

$$\boxed{I \xrightarrow{\text{LoG convolution}} (\nabla^2 G) * I} \tag{61}$$

The edge is identified using a **zero crossing** of the LoG response.

In practice, a threshold is usually applied to avoid treating tiny noise-induced zero crossings as edges.

19 Interpretation of σ in LoG

The scale parameter σ controls the scale of the edges that can be detected.

Small σ:

- less smoothing,
- detects fine details,
- more sensitive to noise.

Large σ:

- stronger smoothing,
- detects broader structures,
- suppresses fine details,
- better noise robustness.

Therefore,

$$\boxed{\sigma \quad \text{controls the scale of edge detection.}} \tag{62}$$

20 From LoG to Canny

The LoG detector improved noise robustness through Gaussian smoothing, but several issues remained.

Typical problems include:

1. Edge localization can be imperfect.
2. Zero-crossing responses can be thick or fragmented.
3. Threshold selection remains important.
4. Multiple scales require repeated filtering.

Canny approached edge detection as an **optimization problem**.

The goal was to design an edge detector satisfying three desirable properties:

1. Good detection.
2. Good localization.
3. Single response to a single edge.

21 Canny Edge Detection: Overview

The standard Canny pipeline is

$$\boxed{\begin{aligned}&\text{Image} \to \text{Gaussian smoothing} \to \text{Gradient}\\ &\qquad \to \text{Non-maximum suppression} \to \text{Double threshold} \to \text{Hysteresis}\end{aligned}} \tag{63}$$

Each stage addresses a particular problem.

Stage	**Purpose**
Gaussian smoothing	Reduce noise
Gradient	Detect intensity changes
Non-maximum suppression	Thin edges
Double threshold	Separate strong/weak edges
Hysteresis	Connect valid weak edges

22 Step 1: Gaussian Smoothing

The input image $I(x, y)$ is first smoothed:

$$\boxed{I_s(x, y) = G(x, y; \sigma) * I(x, y)} \tag{64}$$

where

$$G(x, y; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}. \tag{65}$$

The purpose is to reduce high-frequency noise before differentiation.

23 Step 2: Gradient Computation

Compute

$$I_x = \frac{\partial I_s}{\partial x} \tag{66}$$

and

$$I_y = \frac{\partial I_s}{\partial y}. \tag{67}$$

In practice, finite-difference or derivative kernels such as Sobel are commonly used. The gradient magnitude is

$$\boxed{M(x, y) = \sqrt{I_x^2 + I_y^2}} \tag{68}$$

and the gradient orientation is

$$\boxed{\theta(x, y) = \tan^{-1}\left(\frac{I_y}{I_x}\right)}. \tag{69}$$

24 Step 3: Non-Maximum Suppression

The gradient magnitude often produces thick edges.

Consider the gradient direction θ.

The edge should be a local maximum in the direction perpendicular to the edge, which is the gradient direction.

For each pixel:

1. Determine the gradient direction.
2. Compare the current magnitude with neighboring pixels along the gradient direction.
3. Keep the pixel only if it is a local maximum.

Thus,

$$\boxed{M_{\text{NMS}}(x,y) = \begin{cases} M(x,y), & \text{if local maximum along } \theta, \\ 0, & \text{otherwise.} \end{cases}} \tag{70}$$

This produces thin candidate edges.

25 Gradient Direction Quantization

For efficient implementation, the gradient orientation is often quantized into four principal directions:

$$0^\circ, \quad 45^\circ, \quad 90^\circ, \quad 135^\circ. \tag{71}$$

For example, if

$$\theta \approx 0^\circ,$$

the pixel is compared with its left and right neighbors.

If

$$\theta \approx 90^\circ,$$

it is compared with its upper and lower neighbors.

For 45° and 135°, diagonal neighbors are used.

26 Step 4: Double Thresholding

After non-maximum suppression, two thresholds are applied:

$$T_{\text{high}} \tag{72}$$

and

$$T_{\text{low}}. \tag{73}$$

The pixels are classified as

$$\boxed{E(x,y) = \begin{cases} \text{strong edge}, & M_{\text{NMS}} \geq T_{\text{high}}, \\ \text{weak edge}, & T_{\text{low}} \leq M_{\text{NMS}} < T_{\text{high}}, \\ \text{non-edge}, & M_{\text{NMS}} < T_{\text{low}}. \end{cases}} \tag{74}$$

The two thresholds are important because a single threshold creates a difficult trade-off between missing weak edges and accepting noise.

27 Step 5: Edge Tracking by Hysteresis

A weak edge is not automatically accepted.

Instead, it is accepted if it is connected to a strong edge.

Conceptually,

$$\boxed{\text{Weak edge} + \text{connection to strong edge} \Rightarrow \text{valid edge}} \tag{75}$$

whereas

$$\boxed{\text{Weak edge} + \text{no strong-edge connection} \Rightarrow \text{discard}} \tag{76}$$

This helps preserve genuine contours while eliminating isolated responses caused by noise.

28 Canny: Complete Mathematical Pipeline

The complete Canny procedure can be summarized as follows.

First,

$$I_s = G_\sigma * I. \tag{77}$$

Then,

$$I_x = \frac{\partial I_s}{\partial x}, \qquad I_y = \frac{\partial I_s}{\partial y}. \tag{78}$$

Gradient magnitude:

$$M = \sqrt{I_x^2 + I_y^2}. \tag{79}$$

Gradient direction:

$$\theta = \tan^{-1}\left(\frac{I_y}{I_x}\right). \tag{80}$$

Apply non-maximum suppression:

$$M \to M_{\text{NMS}}. \tag{81}$$

Then double thresholding:

$$M_{\text{NMS}} \to \{\text{strong}, \text{weak}, \text{non-edge}\}. \tag{82}$$

Finally, hysteresis produces the final edge map.

29 How Canny Can Be Understood Mathematically

The Canny detector can be viewed as an optimization problem.

For an ideal step edge corrupted by noise, Canny considered a filter $f(x)$ and sought desirable properties.

The three classical criteria are:

29.1 Good Detection

The filter should maximize the signal-to-noise ratio.

Conceptually,

$$\boxed{\text{maximize} \quad \frac{\text{edge response}}{\text{noise response}}} \tag{83}$$

so that genuine edges are detected reliably.

29.2 Good Localization

The detected edge should be as close as possible to the true edge position.

Thus,

$$\boxed{\text{minimize localization error}} \tag{84}$$

29.3 Single Response

A single physical edge should ideally produce one response.

Thus,

$$\boxed{\text{one edge} \longrightarrow \text{one detector response}} \tag{85}$$

These requirements lead to a filter closely related to the derivative of a Gaussian.

This provides the mathematical motivation for the familiar practical pipeline:

$$\boxed{\text{Gaussian smoothing} + \text{gradient} + \text{NMS} + \text{hysteresis}} \tag{86}$$

30 Relationship Between LoG and Canny

LoG and Canny both use Gaussian smoothing, but their philosophies are different.

	LoG	**Canny**
Smoothing	Gaussian	Gaussian
Derivative	Second derivative	First derivative
Detection criterion	Zero crossing	Local gradient maximum
Edge thinning	Not explicit	Non-maximum suppression
Thresholding	Zero-crossing threshold	Double threshold
Connectivity	Not explicit	Hysteresis
Output	Often thicker	Thin, connected edges

The conceptual progression is

$$\boxed{\text{Derivative} \rightarrow \text{Smooth} + \text{Derivative} \rightarrow \text{Thin} + \text{Connect}} \tag{87}$$

31 Limitations of Classical Edge Detection

Classical methods such as Sobel, Prewitt, LoG and Canny are based on hand-designed mathematical assumptions.

Their main limitations include:

1. Fixed filters are not adaptive to image content.

2. Thresholds often require manual tuning.
3. Texture can generate unwanted edges.
4. Illumination changes can produce false edges.
5. Weak object boundaries may be missed.
6. The distinction between an edge and an object boundary is not always purely local.
7. Natural images contain edges at many spatial scales.

This motivated learning-based edge detection.

32 Evolution of Edge Detection

A useful historical progression is

$$
\boxed{
\begin{array}{c}
\text{Gradient Operators} \\
\Downarrow \\
\text{Sobel, Prewitt, Roberts} \\
\Downarrow \\
\text{Second-Order Operators} \\
\Downarrow \\
\text{LoG / DoG} \\
\Downarrow \\
\text{Optimal Classical Detector} \\
\Downarrow \\
\text{Canny} \\
\Downarrow \\
\text{Hand-Crafted Learning} \\
\Downarrow \\
\text{Structured Edges} \\
\Downarrow \\
\text{Deep CNN Edge Detection} \\
\Downarrow \\
\text{HED, RCF, BDCN, DexiNed} \\
\Downarrow \\
\text{Attention / Transformer-based Edge Detection}
\end{array}
} \tag{88}
$$

33 Structured Edge Detection

Before deep CNNs became dominant, learning-based approaches began learning edge features from data.

Structured Edge Detection learns structured edge representations using random forests.

Instead of predicting only whether an individual pixel is an edge, the method predicts a structured local edge representation.

The key idea is:

$$
\boxed{\text{Hand-designed features} \rightarrow \text{Learned edge representation}} \tag{89}
$$

This was an important transition from fixed operators toward data-driven edge detection.

34 Holistically-Nested Edge Detection (HED)

A major transition occurred with Holistically-Nested Edge Detection (HED).

HED formulates edge detection as an image-to-image prediction problem using a fully convolutional network and deep supervision.

Instead of manually designing an edge filter,

$$\boxed{I \xrightarrow{\text{CNN}} E} \tag{90}$$

where E is a dense edge probability map.

The important idea is **multi-scale feature learning**.

Shallow layers preserve fine spatial details, while deeper layers provide larger receptive fields and semantic information.

HED uses side outputs from multiple network stages and supervises these intermediate predictions.

Thus,

$$\boxed{\text{multi-scale features} + \text{deep supervision} \rightarrow \text{edge map}} \tag{91}$$

HED was introduced by Xie and Tu at ICCV 2015 and demonstrated the effectiveness of deeply supervised, fully convolutional edge prediction.

35 Richer Convolutional Features

Richer Convolutional Features (RCF) further emphasized the importance of combining features from different convolutional layers.

The basic intuition is:

$$\boxed{\text{Shallow features} \rightarrow \text{fine localization}} \tag{92}$$

while

$$\boxed{\text{Deep features} \rightarrow \text{semantic context}} \tag{93}$$

Therefore,

$$\boxed{\text{Fuse features across network depth} \rightarrow \text{better edge detection}} \tag{94}$$

This addresses one of the fundamental difficulties in edge detection: a good detector needs both precise spatial localization and contextual information.

36 Multi-Scale and Encoder–Decoder Edge Detection

Later CNN-based detectors increasingly adopted encoder–decoder architectures and multi-scale processing.

The encoder learns increasingly abstract representations:

$$I \rightarrow F_1 \rightarrow F_2 \rightarrow \cdots \rightarrow F_L. \tag{95}$$

The decoder then combines information from multiple scales:

$$F_L, \ldots, F_2, F_1 \rightarrow E. \tag{96}$$

This allows the detector to combine

$$\boxed{\text{high-level context} + \text{low-level localization.}} \tag{97}$$

Examples include BDCN and other multi-scale CNN architectures.

37 DexiNed

DexiNed introduced a lightweight deep edge detection architecture designed to generate thin edge maps using dense feature extraction and multiple intermediate predictions.

The broader architectural trend is

$$\boxed{\text{Single output} \rightarrow \text{Multi-level outputs} \rightarrow \text{Feature fusion}} \tag{98}$$

This is important because edge information exists at multiple spatial scales.

38 Attention and Transformer-Based Edge Detection

More recent methods have introduced attention and transformer-style architectures.

The motivation is that an edge cannot always be determined from a small local neighborhood.

For example, consider a weak boundary in a cluttered image.

Local information may be ambiguous, but global context may reveal that the boundary belongs to a particular object.

Attention mechanisms allow the network to model long-range dependencies:

$$\boxed{\text{Local appearance} + \text{Global context} \rightarrow \text{better boundary reasoning}} \tag{99}$$

Recent edge-detection research therefore includes:

- CNN-based multi-scale detectors,
- encoder–decoder architectures,
- attention-based architectures,
- transformer-based detectors,
- hybrid CNN–transformer models,
- lightweight real-time edge detectors.

Recent surveys describe this progression from HED and RCF through encoder–decoder, attention and transformer-based approaches.

39 Classical vs. Learning-Based Edge Detection

	Classical	**Learning-based**
Examples	Sobel, Prewitt, LoG, Canny	HED, RCF, BDCN, DexiNed
Features	Hand-designed	Learned
Thresholds	Usually hand-tuned	Learned / data-driven
Context	Mostly local	Local + global
Multi-scale reasoning	Explicitly designed	Learned from hierarchy
Training data	Not required	Usually required
Interpretability	High	Lower
Computation	Usually low	Usually higher
Adaptability	Limited	High

40 A Useful Conceptual View

The development of edge detection can be understood as progressively addressing four problems.

40.1 Problem 1: Detect Intensity Changes

Solution:

$$\boxed{\text{Gradient operators}} \tag{100}$$

Examples:

Roberts, Prewitt, Sobel.

40.2 Problem 2: Suppress Noise

Solution:

$$\boxed{\text{Gaussian smoothing}} \tag{101}$$

Examples:

LoG, Canny.

40.3 Problem 3: Produce Thin and Connected Edges

Solution:

$$\boxed{\text{NMS + hysteresis}} \tag{102}$$

This is the major contribution of the Canny pipeline.

40.4 Problem 4: Understand Which Changes are Meaningful

Solution:

$$\boxed{\text{Learned representations + context}} \tag{103}$$

This motivates CNN-, attention-, and transformer-based edge detectors.

41 Summary

The complete conceptual progression is

$$\boxed{\begin{aligned}&\text{Image} \rightarrow \text{Intensity changes}\\ &\qquad \rightarrow \text{First derivative}\\ &\qquad \rightarrow \text{Sobel / Prewitt}\\ &\qquad \rightarrow \text{Second derivative}\\ &\qquad \rightarrow \text{LoG}\\ &\qquad \rightarrow \text{Canny}\\ &\qquad \rightarrow \text{Learning-based edge detection.}\end{aligned}} \tag{104}$$

The key equations to remember are:

Gradient

$$\nabla I = \begin{bmatrix} I_x \\ I_y \end{bmatrix} \tag{105}$$

Gradient Magnitude

$$\boxed{|\nabla I| = \sqrt{I_x^2 + I_y^2}} \tag{106}$$

Sobel

$$\boxed{G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}} \tag{107}$$

Laplacian

$$\boxed{\nabla^2 I = I_{xx} + I_{yy}} \tag{108}$$

LoG

$$\boxed{LoG = \nabla^2 G} \tag{109}$$

Canny

$$\boxed{\text{Gaussian} \rightarrow \text{Gradient} \rightarrow \text{NMS} \rightarrow \text{Double Threshold} \rightarrow \text{Hysteresis}} \tag{110}$$

The major conceptual shift in modern edge detection is from

$$\boxed{\text{designing the filter}} \tag{111}$$

to

$$\boxed{\text{learning the representation}} \tag{112}$$

and increasingly toward

$$\boxed{\text{learning multi-scale features and global context.}} \tag{113}$$

42 Suggested Teaching Sequence

For a Computer Vision course, the material can be taught in the following order:

1. Introduce edges using a one-dimensional step function.
2. Derive the first derivative and explain why it detects edges.
3. Move from continuous derivatives to finite differences.
4. Derive the Sobel kernel as derivative $\times$ smoothing.
5. Introduce Prewitt as a simpler smoothing-weight choice.
6. Explain why first derivatives are sensitive to noise.
7. Introduce the second derivative and zero crossings.
8. Derive the discrete Laplacian.
9. Introduce Gaussian smoothing.
10. Derive the Laplacian of Gaussian.
11. Explain why LoG detects edges through zero crossings.
12. Introduce Canny as an optimization-based detector.
13. Derive the gradient magnitude and orientation.
14. Explain non-maximum suppression geometrically.
15. Explain double thresholding and hysteresis.
16. Compare Sobel, Prewitt, LoG and Canny.
17. Finish with the transition to learned edge detectors.
18. Introduce HED, RCF and modern multi-scale/attention-based methods as examples of the evolution of the field.

43 Appendix

Non-Maximum Suppression: Mathematical Formulation
After computing the gradient magnitude and direction,

$$M(x,y) = \sqrt{I_x^2(x,y) + I_y^2(x,y)} \tag{114}$$

and

$$\theta(x,y) = \tan^{-1}\left(\frac{I_y(x,y)}{I_x(x,y)}\right), \tag{115}$$

the gradient magnitude image generally contains **thick bands** around edges.

The objective of non-maximum suppression (NMS) is to retain only the pixels that correspond to local maxima of the gradient magnitude **along the gradient direction**.

43.1 Why Compare Along the Gradient Direction?

Recall that the gradient

$$\nabla I = \begin{bmatrix} I_x \\ I_y \end{bmatrix} \tag{116}$$

points in the direction of maximum intensity increase.

The gradient direction is therefore perpendicular to the edge.

Consequently, if a pixel belongs to an edge, its gradient magnitude should be locally maximal in the direction θ.

Thus, NMS performs the following operation:

$$\boxed{\text{Keep a pixel if its gradient magnitude is a local maximum along } \theta} \tag{117}$$

and suppresses it otherwise.

43.2 Idealized Mathematical Formulation

Let

$$\mathbf{u}_\theta = \begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix} \tag{118}$$

be a unit vector in the gradient direction.

For a pixel (x, y), consider the two points located on either side along this direction:

$$\mathbf{p}_+ = \begin{bmatrix} x \\ y \end{bmatrix} + \mathbf{u}_\theta \tag{119}$$

and

$$\mathbf{p}_- = \begin{bmatrix} x \\ y \end{bmatrix} - \mathbf{u}_\theta. \tag{120}$$

The pixel is retained only if

$$\boxed{M(x,y) \geq M(\mathbf{p}_+) \quad \text{and} \quad M(x,y) \geq M(\mathbf{p}_-)} \tag{121}$$

Otherwise, its value is suppressed:

$$\boxed{M_{\text{NMS}}(x,y) = 0.} \tag{122}$$

Therefore,

$$\boxed{M_{\text{NMS}}(x,y) = \begin{cases} M(x,y), & M(x,y) \geq M(\mathbf{p}_+) \text{ and } M(x,y) \geq M(\mathbf{p}_-), \\ 0, & \text{otherwise.} \end{cases}} \tag{123}$$

43.3 Discrete Implementation

The points $\mathbf{p}_+$ and $\mathbf{p}_-$ generally do not coincide with pixel locations.

Therefore, the continuous values are usually approximated by interpolation, or the gradient direction is quantized into a small number of discrete directions.

A common implementation quantizes θ into four directions:

$$\boxed{\theta_q \in \{0^\circ, 45^\circ, 90^\circ, 135^\circ\}.} \tag{124}$$

The corresponding comparisons are:

Gradient direction	Neighbor 1	Neighbor 2
0°	$(x-1, y)$	$(x+1, y)$
45°	$(x-1, y+1)$	$(x+1, y-1)$
90°	$(x, y-1)$	$(x, y+1)$
135°	$(x-1, y-1)$	$(x+1, y+1)$

For example, for $\theta_q = 0^\circ$,

$$M_{\text{NMS}}(x,y) = \begin{cases} M(x,y), & M(x,y) \geq M(x-1,y) \ \wedge \ M(x,y) \geq M(x+1,y), \\ 0, & \text{otherwise.} \end{cases} \tag{125}$$

Thus, a thick gradient response is reduced to approximately one-pixel-wide ridges.

43.4 Interpolation View of NMS

Instead of quantizing the gradient direction, a more accurate implementation can evaluate the gradient magnitude at the exact locations

$$(x + \cos\theta, \ y + \sin\theta) \tag{126}$$

and

$$(x - \cos\theta, \ y - \sin\theta). \tag{127}$$

Because these locations are generally non-integer pixel coordinates, bilinear interpolation can be used.

The NMS condition then becomes

$$\boxed{M(x,y) \geq M(x + \cos\theta, y + \sin\theta)} \tag{128}$$

and

$$\boxed{M(x,y) \geq M(x - \cos\theta, y - \sin\theta).} \tag{129}$$

This formulation makes the geometric meaning of NMS particularly clear: **the gradient magnitude must be locally maximal in the gradient direction**.

44 Double Thresholding

After non-maximum suppression, the remaining pixels form thin candidate edges.

However, weak responses may be caused by noise, texture, or illumination variations.

Canny therefore uses two thresholds:

$$T_{\text{low}} \quad \text{and} \quad T_{\text{high}}, \tag{130}$$

with

$$T_{\text{low}} < T_{\text{high}}. \tag{131}$$

Let

$$M_N(x,y) = M_{\text{NMS}}(x,y). \tag{132}$$

The pixels are divided into three classes:

$$\boxed{C(x,y) = \begin{cases} \text{Strong}, & M_N(x,y) \geq T_{\text{high}}, \\ \text{Weak}, & T_{\text{low}} \leq M_N(x,y) < T_{\text{high}}, \\ \text{NonEdge}, & M_N(x,y) < T_{\text{low}}. \end{cases}} \tag{133}$$

The key idea is that a weak edge is not immediately discarded.

It is retained temporarily because it may represent a genuine continuation of a strong edge.

45 Edge Tracking by Hysteresis

The purpose of hysteresis is to determine which weak edge pixels belong to genuine contours.

The basic principle is:

$$\boxed{\text{Weak edge} + \text{connection to strong edge} \Rightarrow \text{valid edge}} \tag{134}$$

whereas

$$\boxed{\text{Weak edge} + \text{no connection to strong edge} \Rightarrow \text{discard}} \tag{135}$$

This prevents isolated weak responses from being interpreted as edges while preserving weak portions of genuine contours.

45.1 Graph Interpretation of Hysteresis

Hysteresis can be understood naturally as a graph traversal problem.

Let each candidate edge pixel be a node in a graph.

Two pixels are connected if they are neighboring pixels, typically using 8-connectivity:

$$\mathcal{N}_8(x,y) = \{(i,j) : |i-x| \leq 1,\ |j-y| \leq 1,\ (i,j) \neq (x,y)\}. \tag{136}$$

Let

$$S = \{(x,y) : M_N(x,y) \geq T_{\text{high}}\} \tag{137}$$

be the set of strong edge pixels, and

$$W = \{(x,y) : T_{\text{low}} \leq M_N(x,y) < T_{\text{high}}\} \tag{138}$$

be the set of weak edge pixels.

The final edge set is the subset of weak pixels that can be connected to at least one strong pixel through a path consisting entirely of strong or weak edge pixels.

Formally,

$$\boxed{E_{\text{final}} = S \cup \{p \in W : \exists \text{ a path from } p \text{ to } S \text{ using only } S \cup W\}.} \tag{139}$$

This is the mathematical essence of edge tracking by hysteresis.

45.2 Recursive Formulation

The same idea can be expressed recursively.

Let $E^{(0)} = S$ be the initial set of strong edges.

At iteration $k+1$, add weak pixels that are connected to the currently accepted edge set:

$$E^{(k+1)} = E^{(k)} \cup \left\{ p \in W : \mathcal{N}_8(p) \cap E^{(k)} \neq \emptyset \right\}. \tag{140}$$

The process continues until no new weak pixels can be added:

$$E^{(k+1)} = E^{(k)}. \tag{141}$$

The converged set is the final Canny edge map:

$$\boxed{E_{\text{final}} = \lim_{k \to \infty} E^{(k)}.} \tag{142}$$

In practice, this is implemented efficiently using a stack, queue, or recursive/iterative connected-component traversal starting from the strong edge pixels.

46 Why Hysteresis is Better Than a Single Threshold

Suppose a real object boundary contains a range of gradient magnitudes:

$$M_1 > M_2 > M_3 > M_4. \tag{143}$$

A single high threshold might retain only

$$M_1 \tag{144}$$

and break the contour.

A low threshold, on the other hand, may accept noise.

Hysteresis solves this problem by using

$$T_{\text{low}} < T_{\text{high}}. \tag{145}$$

A strong response provides an "anchor", while weaker connected responses are allowed to continue the contour.

Thus,

$$\boxed{\text{High threshold controls reliability}} \tag{146}$$

while

$$\boxed{\text{Low threshold controls continuity.}} \tag{147}$$

47 NMS and Hysteresis: Their Different Roles

It is useful to distinguish the two operations.

	Non-Maximum Suppression	**Hysteresis**
Input	Gradient magnitude	Thresholded edge candidates
Main purpose	Thin edges	Connect valid edges
Uses gradient direction?	Yes	No
Uses neighborhood connectivity?	Local directional comparison	Yes
Main question	"Is this pixel a local maximum?"	"Is this weak pixel connected to a strong edge?"
Result	Thin candidate edges	Final connected edge map

Therefore, the two stages solve fundamentally different problems:

$$\boxed{\text{NMS} \rightarrow \text{spatial localization}} \tag{148}$$

whereas

$$\boxed{\text{Hysteresis} \rightarrow \text{edge connectivity.}} \tag{149}$$

48 Complete Canny Pipeline with Mathematical Operations

The complete mathematical pipeline can now be written as

$$I \xrightarrow{G_\sigma *} I_s \tag{150}$$

followed by

$$I_x = \frac{\partial I_s}{\partial x}, \qquad I_y = \frac{\partial I_s}{\partial y}, \tag{151}$$

$$M = \sqrt{I_x^2 + I_y^2}, \qquad \theta = \tan^{-1}\left(\frac{I_y}{I_x}\right), \tag{152}$$

then

$$M \xrightarrow{\text{NMS}} M_N, \tag{153}$$

followed by

$$M_N \xrightarrow{\text{double threshold}} \{S, W, N\}, \tag{154}$$

and finally

$$\boxed{\{S, W, N\} \xrightarrow{\text{hysteresis}} E_{\text{final}}.} \tag{155}$$

Thus, Canny can be viewed as solving three increasingly refined problems:

$$\boxed{\begin{array}{rl} \text{Gradient :} & \text{Where is intensity changing?} \\ \text{NMS :} & \text{Where is the change maximally localized?} \\ \text{Hysteresis :} & \text{Which localized responses form real contours?} \end{array}} \tag{156}$$